This idea of wrapping a new feature around an existing method is common in Rails. In all cases, you end up with three methods that follow the naming conventions I just showed you: method, method_with_feature, and method_without_feature. The only the first two methods include the new feature.

Instead of duplicating these aliases all over the place, Rails provided a metaprogramming method that did it all for you. It was named Module#alias_method_chain, and it was part of the Active Support library. I'm saying "it was" rather than "it is" for reasons that will be clear soon—but if you look inside Active Support, you'll find alias_method_chain is still there. Let's look at it.

Inside alias_method_chain

Here is the code of alias_method_chain:

gems/activesupport-4.1.0/lib/active_support/core_ext/module/aliasing.rb

```
class Module
  def alias_method_chain(target, feature)
    # Strip out punctuation on predicates or bang methods since
    # e.g. target?_without_feature is not a valid method name.
    aliased_target, punctuation = target.to_s.sub(/([?!=])$/, ''), $1
    yield(aliased_target, punctuation) if block_given?

    with_method = "#{aliased_target}_with_#{feature}#{punctuation}"
    without_method = "#{aliased_target}_without_#{feature}#{punctuation}"

    alias_method without_method, target
    alias_method target, with_method

    case
    when public_method_defined?(without_method)
      public target
    when protected_method_defined?(without_method)
      protected target
    when private_method_defined?(without_method)
      private target
    end
  end
end
```

alias_method_chain takes the name of a target method and the name of an additional feature. From those two, it calculates the name of two new methods: target_without_feature and target_with_feature. Then it stores away the original target as target_without_feature, and it aliases target_with_feature to target (assuming that a method called target_with_feature is defined somewhere in the same module). Finally, the case switch sets the visibility of target_without_feature so that it's the same visibility as the original target.